App Visitas

Matriz de brechas y contratos de integración backend

Documento funcional/técnico para equipo backend — v2.0

1. Objetivo

Este documento resume, de forma comparativa, lo que el backend actualmente expone y lo que la App Visitas necesita para soportar el flujo real implementado en frontend: promotor, coordinador, supervisor/especialista y director.

Además de la matriz de brechas, este documento incluye contratos JSON concretos para los endpoints críticos, tabla oficial de equivalencia de roles y especificación detallada del sistema de notificaciones FCM. El objetivo es que el equipo backend pueda actuar directamente sin requerir reuniones adicionales de aclaración.

---

2. Resumen ejecutivo

- El backend entregado cubre autenticación JWT, gestión básica de usuarios y una visita simple con una sola entrada/evidencia. Sin embargo, debe alinearse a los únicos roles válidos de la app: PEC/PROMOTOR, COORDINADOR, ESPECIALISTA/SUPERVISOR y DIRECTOR.
- La app necesita un modelo multirol, con estados separados por promotor y coordinador, revisión del supervisor y vistas/reportes para director.
- Existen brechas funcionales y de modelo de datos que backend debe cerrar antes de integrar completamente el frontend actual.
- La tabla central de este documento especifica exactamente qué existe hoy, qué necesita la app y qué acción se requiere.

2.1 Endpoints validados como funcionales

Endpoint	Estado	Comentario
POST /auth/login	✅ **Usable**	Devuelve JWT válido. Punto de entrada para integración inmediata.
GET /visitas	✅ **Usable**	Lista visitas con Bearer Token. Base para dashboards.
GET /visitas/{id}	✅ **Usable**	Devuelve detalle simple de visita.
POST /visitas/{id}/registrar-entrada	⚠ **Parcial**	Solo soporta una entrada embebida. Debe dividirse en endpoints por actor.
PUT /users/{id}	✅ **Usable**	Actualiza nombres, apellidos y perfil.

Endpoint	Estado	Comentario
PUT /visitas/{id}	✕ **No usable**	Responde 400: exige perfil PROFESOR. Este perfil no debe existir en la app; backend debe reemplazarlo por PEC/PROMOTOR, COORDINADOR, ESPECIALISTA/SUPERVISOR o DIRECTOR según corresponda.
GET /users/profesores	✕ **Error 500**	Falla internamente con token válido. Requiere corrección urgente.

3. Tabla oficial de equivalencia de roles

La app usa únicamente cuatro roles funcionales que determinan navegación, dashboard, historial, detalle y flujos de negocio: PEC (Promotor), Coordinador, Especialista (Supervisor) y Director. Cualquier rol adicional, especialmente PROFESOR, no debe existir ni ser usado por la app. Esta es la equivalencia propuesta que debe implementarse en el enum de perfiles del backend:

Rol en la app	Rol propuesto en backend	Estado actual	Acción requerida
PEC (Promotor)	**PEC / PROMOTOR**	✕ **No existe**	Agregar PEC/PROMOTOR al enum de perfiles. No usar PROFESOR.
Coordinador	**COORDINADOR**	⚠ **Parcial**	Existe pero la visita no refleja el flujo completo del coordinador.
Especialista (Supervisor)	**ESPECIALISTA / SUPERVISOR**	⚠ **ADMIN temp.**	Reemplazar uso temporal de ADMIN por rol ESPECIALISTA/SUPERVISOR oficial.
director	**DIRECTOR**	✅ **Existe**	Alineado en auth. Faltan endpoints de overview y reportes.
— (interno backend)	**PROFESOR**	✕ **Inconsistente**	No debe existir para la app. Debe eliminarse o mapearse correctamente a uno de los cuatro roles oficiales: PEC/PROMOTOR, COORDINADOR, ESPECIALISTA/SUPERVISOR o DIRECTOR.

4. Matriz comparativa de modelos y endpoints

Modelo / Endpoint	Backend entregado	App necesita	Acción requerida
AUTENTICACIÓN Y USUARIOS			

Modelo / Endpoint	Backend entregado	App necesita	Acción requerida
Modelo User / sesión	RegisterRequestDto, LoginRequestDto, AuthResponseDto, UserResponseDto. Campos: usuario, nombres, apellidos, perfil, token.	id, username, fullName, role, schoolName, modularCode, ugelName, locationName, documentNumber, birthDate.	Ampliar UserResponseDto / AuthResponseDto con todos los campos requeridos.
GET /auth/me	No existe.	Recuperar usuario autenticado sin rehacer login.	Crear endpoint protegido GET /auth/me que devuelva el perfil completo.
MODELO DE VISITA			
Modelo VisitSchedule	VisitResponseDto con: id, profesorId, lugar, fechaHora, fechaHoraEntrada, latitudEntrada, longitudEntrada, observacionEntrada, evidenciaRuta.	coordinatorId, promoterId, supervisorId, coordinatorName, promoterName, placeName, scheduledDate, expectedStartTime, expectedEndTime, promoterArrivalConfirmedAt, promoterVisitClosedAt, coordinatorCheckInAt, coordinatorVisitClosedAt, notificationSentToSupervisor.	Rediseñar o ampliar el modelo de visita. Ver contrato JSON en sección 5.1.
GET /visitas	Existe. Lista visitas simples con token.	Listado filtrable por rol: visitas del promotor, del coordinador, y todas para supervisor/director.	Mantener y agregar filtros por rol (query param ?role=promotor o endpoints especializados).
GET /visitas/{id}	Existe. Devuelve detalle simple.	Detalle completo multirol con estados por actor.	Ampliar response con los campos del nuevo modelo. Ver contrato JSON en sección 5.1.
POST /visitas	Existe. Crea visita con profesorId, lugar y fechaHora; este campo debe reemplazarse porque la app no maneja PROFESOR.	Crear visitas con coordinatorId, promoterId, supervisorId, fecha, rango horario y coordenadas base.	Redefinir request de creación. Ver contrato JSON en sección 5.2.

Modelo / Endpoint	Backend entregado	App necesita	Acción requerida
PUT /visitas/{id}	Existe pero responde 400: exige perfil PROFESOR. Este rol no forma parte de la app y debe eliminarse de la lógica funcional.	Actualizar visita de forma consistente con el nuevo modelo.	Corregir endpoint y alinear al nuevo modelo. Prioridad alta.
ENDPOINTS OPERATIVOS POR ACTOR			
PATCH /visitas/{id}/promoter-arrival	No existe.	Confirmar llegada del promotor. Requiere: latitude, longitude, observation, evidenciaBase64.	Crear endpoint. Setea promoterArrivalConfirmedAt. Ver contrato en sección 5.3.
PATCH /visitas/{id}/promoter-close	No existe.	Cerrar visita desde el flujo del promotor.	Crear endpoint. Setea promoterVisitClosedAt. Trigger: chequear si coordinador ya cerró para disparar notificación al supervisor.
PATCH /visitas/{id}/coordinator-checkin	No existe.	Registrar entrada del coordinador. Requiere: latitude, longitude, observation, evidenciaBase64.	Crear endpoint. Setea coordinatorCheckInAt. Ver contrato en sección 5.3.
PATCH /visitas/{id}/coordinator-close	No existe.	Cerrar visita desde el flujo del coordinador.	Crear endpoint. Setea coordinatorVisitClosedAt. Trigger: chequear si promotor ya cerró para disparar notificación al supervisor.
EVIDENCIAS			
Modelo VisitEvidence	No existe como entidad independiente. Evidencia embebida en la visita.	id, visitId, userId, userRole, eventType, imageUrl, latitude, longitude, observation, createdAt.	Crear entidad/DTO independiente de evidencias. Ver contrato en sección 5.4.
POST /visit-evidences	No existe.	Guardar evidencia para coordinator_check_in y promoter_arrival_confirmation.	Crear endpoint para registrar evidencias de forma independiente.
GET /visit-evidences/visit/{visitId}	No existe.	Recuperar todas las evidencias de una visita para comparación del supervisor.	Crear endpoint de listado de evidencias por visita.
REVISIÓN DEL SUPERVISOR			

Modelo / Endpoint	Backend entregado	App necesita	Acción requerida
Modelo VisitReview	No existe.	visitId, supervisorId, finalStatus (Conforme/Observada), comment, reviewedAt.	Crear entidad/DTO de revisión. Ver contrato en sección 5.5.
POST /visit-reviews	No existe.	Supervisor marca la visita como Conforme u Observada con comentario.	Crear endpoint. Ver contrato en sección 5.5.
GET /visit-reviews/visit/{visitId}	No existe.	Consultar si una visita ya fue revisada y cuál fue el resultado.	Crear endpoint de revisión por visita.
GET /visitas/{id}/validation	No existe. Solo se guardan coordenadas en la entrada.	Comparar coordenadas de promotor y coordinador: distanceInMeters, isConsistent, status, message.	Crear endpoint de validación GPS o incluir resultado en el detalle de la visita.
DIRECTOR			
GET /director/overview	No existe.	KPIs, resumen por coordinador, por zona, alertas y tendencia.	Crear DTO y endpoint. Ver contrato en sección 5.6.
GET /director/reports	No existe.	Reportes filtrables por fecha, coordinador, promotor, estado y zona. Incluye cumplimiento e inconsistencias.	Crear DTO y endpoint. Ver contrato en sección 5.7.
NOTIFICACIONES FCM			
POST /devices/fcm-token	No existe.	Registrar token FCM del dispositivo del supervisor para notificaciones push.	Crear DeviceToken y endpoint. Ver especificación completa en sección 6.

5. Contratos JSON esperados por la app

Esta sección especifica exactamente la estructura que la app espera recibir o enviar en cada endpoint crítico. Los nombres de campos deben respetarse tal como están definidos aquí para evitar mapeos adicionales en el frontend.

5.1 GET /visitas/{id} — Respuesta esperada

El detalle de una visita debe incluir los estados de cada actor por separado. Los campos de timestamp son null mientras el actor no haya completado esa acción.

Response 200 OK:

```
{
  "id": "v1",
  "coordinatorId": "u2",
  "promoterId": "p1",
  "supervisorId": "u3",
  "coordinatorName": "Lic. Sadith Carmin Ruiz Huaynacari",
  "promoterName": "Jadrian Cahuachi del Aguila",
  "placeName": "Los Gusanitos - 1° de Octubre",
  "scheduledDate": "2026-04-12",
  "expectedStartTime": "08:00",
  "expectedEndTime": "09:00",
  "latitude": -4.2710,
  "longitude": -69.9390,
  "status": "Programada",
  "promoterArrivalConfirmedAt": null,
  "promoterVisitClosedAt": null,
  "coordinatorCheckInAt": null,
  "coordinatorVisitClosedAt": null,
  "notificationSentToSupervisor": false
}
```

5.2 POST /visitas — Request de creación

Request body:

```
{
  "coordinatorId": "u2",
  "promoterId": "p1",
  "supervisorId": "u3",
  "placeName": "Los Gusanitos - 1° de Octubre",
  "scheduledDate": "2026-04-12",
  "expectedStartTime": "08:00",
  "expectedEndTime": "09:00",
  "latitude": -4.2710,
  "longitude": -69.9390
}
```

5.3 Endpoints operativos por actor

Cada endpoint de acción del promotor o coordinador debe aceptar el siguiente cuerpo. El campo evidenciaBase64 es opcional si las evidencias se registran por separado en /visit-evidences.

PATCH /visitas/{id}/promoter-arrival — Request:

```
{
  "latitude": -4.2715,
  "longitude": -69.9388,
  "observation": "Llegué al local puntualmente.",
  "evidenciaBase64": "..."
}
```

PATCH /visitas/{id}/promoter-arrival — Response 200 OK:

```
{
  "visitId": "v1",
  "promoterArrivalConfirmedAt": "2026-04-12T08:05:00Z",
  "message": "Llegada del promotor confirmada correctamente."
}
```

Los endpoints /promoter-close, /coordinator-checkin y /coordinator-close siguen la misma estructura de request. El response de /promoter-close y /coordinator-close debe incluir además:

Campo adicional en response de cierre:

```
{
  "notificationSentToSupervisor": true
  // true si ambos actores ya cerraron y se disparó la notificación
}
```

5.4 Modelo de evidencia independiente

Si las evidencias se implementan como recurso independiente (recomendado), este es el contrato esperado.

POST /visit-evidences — Request:

```
{
  "visitId": "v1",
  "userId": "p1",
  "userRole": "promotor",
  "eventType": "promoter_arrival_confirmation",
  "imageBase64": "...",
  "latitude": -4.2715,
  "longitude": -69.9388,
  "observation": "Llegué al local puntualmente."
}
```

GET /visit-evidences/visit/{visitId} — Response 200 OK:

```
[
  {
    "id": "ev1",
    "visitId": "v1",
    "userId": "u2",
    "userRole": "coordinator",
    "eventType": "coordinator check in",
    "imageUrl": "https://storage.example.com/evidencias/ev1.jpg",
    "latitude": -4.2710,
    "longitude": -69.9390,
    "observation": "Check-in registrado correctamente.",
    "createdAt": "2026-04-12T08:02:00Z"
  },
  {
    "id": "ev2",
    "visitId": "v1",
    "userId": "p1",
    "userRole": "promotor",
    "eventType": "promoter arrival confirmation",
    "imageUrl": "https://storage.example.com/evidencias/ev2.jpg",
    "latitude": -4.2715,
    "longitude": -69.9388,
    "observation": "Llegué al local puntualmente.",
    "createdAt": "2026-04-12T08:05:00Z"
  }
]
```

5.5 Revisión del supervisor

POST /visit-reviews — Request:

```
{
  "visitId": "v1",
  "supervisorId": "u3",
  "finalStatus": "Conforme",
  "comment": "Evidencias coherentes. Coordenadas dentro del rango."
}
```

POST /visit-reviews — Response 201 Created:

```
{
  "id": "rev1",
  "visitId": "v1",
  "supervisorId": "u3",
  "finalStatus": "Conforme",
  "comment": "Evidencias coherentes. Coordenadas dentro del rango.",
  "reviewedAt": "2026-04-12T10:30:00Z"
}
```

GET /visit-reviews/visit/{visitId} — Response 200 OK:

```
{
  "id": "rev1",
  "visitId": "v1",
  "supervisorId": "u3",
  "finalStatus": "Conforme",
  "comment": "Evidencias coherentes. Coordenadas dentro del rango.",
  "reviewedAt": "2026-04-12T10:30:00Z"
}
// 404 Not Found si la visita aún no fue revisada
```

5.6 GET /director/overview — Panorama general

Response 200 OK:

```
{
  "totalProgramadas": 18,
  "totalRealizadas": 13,
  "totalValidadas": 10,
  "totalObservadas": 3,
  "totalPendientes": 5,
  "porcentajeCumplimiento": 72.0,
  "coordinadores": [
    {
      "nombre": "Lic. Sadith Carmin Ruiz Huaynacari",
      "programadas": 10,
      "completadas": 8,
      "conformes": 6,
      "observadas": 2
    }
  ],
  "zonas": [
    { "nombre": "Ramón Castilla", "programadas": 8, "realizadas": 5 }
  ],
  "alertas": [
    { "titulo": "Visitas observadas", "detalle": "3 visitas marcadas como observadas." }
  ],
  "tendencia": {
    "cumplimientoActual": 72.0,
    "cumplimientoSemanaAnterior": 64.0,
    "observadasActuales": 3,
    "pendientesActuales": 5
  }
}
```

5.7 GET /director/reports — Reportes filtrables

Este endpoint debe soportar los siguientes query params opcionales: fechaDesde, fechaHasta, coordinadorId, promotorId, estado (Realizada/Pendiente/Observada), zona.

Response 200 OK:

```
{
  "visitas": [
    {
      "fecha": "16/04/2026",
      "coordinador": "Lic. Sadith Carmin Ruiz Huaynacari",
```

```
      "promotor": "Jadrian Cahuachi del Aguila",
      "lugar": "Los Gusanitos - Ramón Castilla",
      "estado": "Realizada",
      "validacionFinal": "Conforme",
      "observacionSupervisor": "Evidencias coherentes."
    }
  ],
  "cumplimientoPorCoordinador": [
    {
      "nombre": "Lic. Sadith Carmin Ruiz Huaynacari",
      "programadas": 10,
      "realizadas": 8,
      "conformes": 6,
      "observadas": 2
    }
  ],
  "inconsistencias": [
    {
      "titulo": "Diferencia GPS",
      "cantidad": 1,
      "detalle": "Diferencia geográfica superior al rango permitido."
    }
  ]
}
```

6. Especificación de notificaciones FCM al supervisor

6.1 Trigger del lado del servidor

La notificación debe dispararse automáticamente cuando se cumpla esta condición en la misma visita:

- promoterVisitClosedAt != null
- coordinatorVisitClosedAt != null
- notificationSentToSupervisor == false

El trigger debe evaluarse cada vez que se actualice promoterVisitClosedAt o coordinatorVisitClosedAt. Si ambas condiciones se cumplen, enviar la notificación FCM al token del supervisor asignado a esa visita (campo supervisorId) y marcar notificationSentToSupervisor = true para evitar duplicados.

6.2 Registro del token FCM

POST /devices/fcm-token — Request:

```
{
  "userId": "u3",
  "fcmToken": "fME_xyz123...",
  "platform": "android"
}
```

POST /devices/fcm-token — Response 200 OK:

```
{
  "message": "Token FCM registrado correctamente."
}
```

6.3 Payload de la notificación push

El servidor debe enviar al token FCM del supervisor el siguiente payload:

FCM notification payload:

```
{
  "to": "<fcm_token_del_supervisor>",
  "notification": {
    "title": "Visita lista para revisar",
    "body": "<promoterName> · <placeName>"
  },
  "data": {
    "visitId": "v1",
    "type": "VISIT_READY_FOR_REVIEW"
  }
}
```

El campo data.visitId permite que la app móvil navegue directamente a la pantalla de comparación de evidencias al recibir la notificación.

---

7. Prioridad recomendada de implementación

Fase	Qué implementar	Qué puede esperar / seguir en mock
P1 Inmediata	Auth real alineada a roles. Corrección de PUT /visitas/{id} y GET /users/profesores. Modelo de visita multirol. Endpoints PATCH por actor (promoter-arrival, promoter-close, coordinator-checkin, coordinator-close).	Evidencias independientes, revisión supervisor, overview director, FCM.
P2 Transitoria	Evidencias como recurso independiente (POST y GET /visit-evidences). Revisión del supervisor (POST y GET /visit-reviews). Validación GPS automática. GET /auth/me.	Overview y reportes director, FCM.
P3 Futura	Overview y reportes del director (GET /director/overview, GET /director/reports). Registro de token FCM y lógica server-side de notificaciones push.	Nada crítico si P1 y P2 están completos.

---

8. Conclusión

El backend entregado es una buena base técnica, pero actualmente responde a un caso más simple que la App Visitas. Para integrarlo con el frontend ya desarrollado, backend debe evolucionar desde un modelo de "visita simple con una sola entrada/evidencia" hacia un modelo multirol con promotor, coordinador, supervisor/especialista y director.

Los contratos JSON incluidos en la sección 5 están directamente alineados con las entidades que la app ya usa internamente. Si el backend respeta exactamente esos contratos, la integración desde frontend requerirá únicamente implementar los mappers de datos sin modificar pantallas ni lógica de negocio existente.

Solicito al equipo backend la revisión y atención de las brechas detalladas en este documento siguiendo el orden de prioridades definido en la sección 7.